Vector-Shield: Master Project Handoff & System Guide

Sub-2ms Dual-Stage Neuro-Symbolic Middleware for Defending Agentic RAG Against Indirect Prompt Injections

Project: Vector-Shield (AI Security Middleware)	**Institution:** Amrita Vishwa Vidyapeetham
Lead Architect: Vignesh V S (cb.ai.u4aid23057)	**Department:** School of AI & Data Science
Team Members: Karthick Ruban, Sanjay Senthil, Hari Prakash	**Supervisor:** Dr. Lekshmi CR
Repository: github.com/VigneshVS2005/Vectorshield	**Live Cockpit:** vectorshield.streamlit.app

1. Executive Summary & Plain-Language Explanation

What is this project? Vector-Shield is an enterprise cybersecurity firewall placed between an AI's memory (Vector Database) and the AI's brain (LLM). In modern enterprise AI systems (Retrieval-Augmented Generation / RAG), attackers don't attack the chat box directly. Instead, they hide malicious instructions inside vendor invoices, resumes, or emails (Indirect Prompt Injection). When an innocent employee queries the system, the vector database fetches that poisoned chunk, and the AI executes the attacker's commands (such as wire fraud or credential theft).

The Analogy (The King and The Food Taster): The LLM is a King who reads scrolls from a library. The attacker slips a poisoned scroll into the archives. Vector-Shield acts as the Royal Food Taster: before the King touches any scroll, Vector-Shield scans its chemical signature (vector density) in **1.4 milliseconds**. If it smells like poison, Vector-Shield burns the scroll, grabs a clean backup, and the King answers the citizen safely.

2. Why Existing GenAI Defenses Fail & Novel Contributions

Examiners frequently ask: *'Why not just use Llama Guard, NeMo Guardrails, or OpenAI Moderation?'* The table below outlines the core technical defense:

Dimension	Standard LLM Guards (Llama Guard / NeMo)	Vector-Shield (Our System)
Inspection Layer	Text Generation Layer (calls secondary LLM)	High-Dimensional Vector Space (Pre-LLM)
Latency Added	800 ms to 2,500 ms per chunk (Destroys UX)	1.2 ms to 1.8 ms per vector (1,000x faster)
Token Cost	High ($$$ pay tokens for every chunk)	Zero Token Cost ($0 purely local math)
Evasion Resistance	Bypassed by Base64, leetspeak, hypotheticals	Catches dense semantic clustering anomalies
Failure Recovery	Hard Refusal: 'I cannot answer this'	Autonomous Self-Healing LangGraph Rewind

Key Research Novelties:

1. Cosine Domain-Centroid Density Scaling: Standard Euclidean LOF fails in high-dimensional sentence spaces due to the Curse of Dimensionality (all normalized vectors have distance approx sqrt(2)). We developed an angular scaling formula combining k-NN density with cosine deviation from the domain centroid.

2. Sub-2ms In-Retrieval Quarantine: Neutralizes payloads before prompt synthesis, eliminating GPU memory leaks and prompt hijacking.

3. Stateful Graph Recovery: Instead of hard errors, compiled LangGraph loops purge poisoned candidates and re-retrieve clean context.

3. Productization Architecture & Streamlit UI Guide

Vector-Shield is engineered as a commercial enterprise B2B middleware. It deploys as a containerized sidecar or python package (pip install vector-shield) between client vector databases and LLM backends.

Streamlit Security Cockpit Tabs:

• **Tab 1: Live Attack Simulator:** Allows testing corporate queries vs. Garak injections with a live Toggle Shield switch demonstrating exploit execution vs. secure sanitization.

• **Tab 2: Stage 2 ModernBERT Inspection:** Deep token-level sequence classification (149M params) resolving borderline ambiguity (1.30 < LOF < 2.50) with probability gauges.

• **Tab 3: Dynamic Doc/PDF Ingestion Studio:** Live ingestion engine that embeds, checks LOF density, and blocks poisoned documents at ingestion time before they reach the database.

• **Tab 4: 3D Vector Hyperspace:** Interactive 3D PCA scatter plot showing clean corporate clusters (green) vs. outlier injection vectors (red).

• **Tab 5: Architecture & SaaS Topology:** Mathematical formulations and enterprise deployment diagrams.

• **Tab 6: SIEM Webhook & Telemetry:** Integration with Splunk, Datadog, Slack, and Syslog with real-time incident auditing.

4. NVIDIA A100 GPU Cluster Training Guide

To train the Stage 2 ModernBERT neural classifier on your A100 GPU node, run the following commands directly inside your JupyterLab terminal:

```
# 1. Pull the repository
cd Vectorshield
git pull origin main

# 2. Install dependencies
pip install -r requirements.txt
pip install accelerate

# 3. Run scaled corpus generation
python src/dataset_scaler.py

# 4. Launch A100 GPU Training (Takes ~2 minutes)
python src/train_stage2_modernbert.py --batch_size 16 --epochs 3 --fp16
```

5. Master AGY Prompt for Headless Execution on A100

When you log into your A100 JupyterLab node and start the `agy` CLI, copy and paste the master prompt below. It will autonomously execute data generation, train the model, verify checkpoint weights, run pytest suites, and push results back to GitHub.

```
AGY MASTER DIRECTIVE: You are running on an NVIDIA A100 GPU node inside the Vectorshield repository.
Execute the following sequence autonomously without stopping:
1. Verify GPU availability: python -c 'import torch; assert torch.cuda.is_available();
print(torch.cuda.get_device_name(0))'
2. Run dataset scaling: python src/dataset_scaler.py
3. Train Stage 2 ModernBERT: python src/train_stage2_modernbert.py --batch_size 16 --epochs 3 --fp16
```

```
4. Verify saved weights in models/stage2_modernbert/
5. Run full test suite: pytest tests/
6. Stage, commit, and push updated weights/metrics to GitHub origin main: git add . && git commit -m
'feat: complete Stage 2 ModernBERT fine-tuning on A100' && git push origin main
Provide final benchmark telemetry upon completion.
```